

# Independent audit record

Repository source: `certificate/INDEPENDENT_AUDIT.md`

## Contents

|                                                |   |
|------------------------------------------------|---|
| Scope . . . . .                                | 1 |
| Audit matrix . . . . .                         | 1 |
| Binary64 Python audit . . . . .                | 2 |
| Base-R audit . . . . .                         | 2 |
| Reconstruction and regression tests . . . . .  | 2 |
| Adversarial mutation campaign . . . . .        | 3 |
| Construction-time export equivalence . . . . . | 3 |
| Paper and documentation audits . . . . .       | 3 |
| Independence limitations . . . . .             | 3 |
| External audit still requested . . . . .       | 4 |

## Scope

This record describes the audit layers that are independent of the exact rational verification path to varying degrees. It does not call them independent mathematical review.

The exact verifier is the finite proof authority. The audits are intended to catch transcription, data-shape, arithmetic, assembly, and implementation errors through separately organized computations.

## Audit matrix

| Layer                   | Program or record                               | Arithmetic                                | Shares with exact path                                | Current result           |
|-------------------------|-------------------------------------------------|-------------------------------------------|-------------------------------------------------------|--------------------------|
| Exact verifier          | <code>certify_universal_bound.py</code>         | Python integers and <code>Fraction</code> | Authoritative proof objects                           | PASS                     |
| Binary64 reconstruction | <code>audit_universal_bound.py</code>           | NumPy binary64                            | Reads audit renderings and common immutable data      | PASS                     |
| Base-R reconstruction   | <code>audit_universal_bound.R</code>            | Base-R numerical arithmetic               | Reads archived exact/decimal bridge data              | PASS on user machine     |
| Reconstruction tests    | <code>test_universal_bound.py</code>            | Exact and binary64                        | Calls production code and rebuilds objects            | 10/10 PASS               |
| Mutation tests          | <code>test_adversarial_mutation.py</code>       | Exact production gates                    | Temporary mutated copies                              | 7/7 rejected as required |
| Export equivalence      | <code>transcripts/export_equivalence.txt</code> | Semantic field comparison                 | Private source and public export at construction time | 10 critical fields match |
| Paper reconciliation    | <code>paper/reconcile_certificate.py</code>     | Exact parsing and byte comparison         | Authoritative JSON and generated TeX                  | PASS                     |

| Layer                    | Program or record                              | Arithmetic                    | Shares with exact path      | Current result             |
|--------------------------|------------------------------------------------|-------------------------------|-----------------------------|----------------------------|
| Exact-constants appendix | <code>proof/generate_exact_constants.py</code> | Deterministic JSON projection | Authoritative JSON          | CURRENT                    |
| Release-document check   | <code>ci/verify_release_docs.py</code>         | Structural and textual checks | Public repository documents | PASS after this checkpoint |

## Binary64 Python audit

`audit_universal_bound.py` is organized as a numerical reconstruction rather than a wrapper around the exact theorem function. It reports, among other quantities:

```

fan vertices:          98306
changed fan vertices:  18229
minimum binary64 fan margin: 2.888902546002714e-10
sharp determinant squared: 0.990112274299406
core volume lower:    0.411735710312777
three-pair cap lower: 0.000143960902055
near-Jung lower:      0.411879671214832
universal coefficient: 0.411877563780302
MINVOL UNIVERSAL CONTACT-BOUND BINARY64 AUDIT: PASS

```

Its agreement with the exact path is evidence against many ordinary implementation errors. It cannot certify a strict inequality whose margin is smaller than uncontrolled floating-point error, so it is not the proof authority.

## Base-R audit

`audit_universal_bound.R` uses base R only and reconstructs the principal lower-row, fan, determinant, coordinate-width, cap, disjointness, and assembly quantities from the public audit records. The named author's local transcript ends with:

```
MINVOL UNIVERSAL CONTACT-BOUND BASE-R AUDIT: PASS
```

Selected reported values include:

```

fan orbit rows / vertices: 24833/98306
minimum fan margin:       2.888904207092860e-10
determinant squared lower: 0.990112274299406
core volume lower:        0.411735710312777
three-pair cap lower:     0.000143960902055
near-Jung volume lower:   0.411879671214832
universal coefficient lower: 0.411877563780302

```

The archived transcript is `transcripts/base_r_audit.txt`. The construction container does not include `Rscript`, so the R path was not rerun there. This limitation is recorded rather than hidden.

## Reconstruction and regression tests

The ten Python tests cover:

- the binary64 audit;
- cap allocations and cross-axis disjointness;
- certificate claim fields;
- coordinate-width sharpening;
- byte-identical exact reconstruction;
- fan reconstruction and reference regression;
- reference-fan fingerprinting;
- gap-free refined lower rows;

- sharp determinant census; and
- absence of private-path dependencies.

The archived unit-test transcript is `transcripts/python_unittest.txt`.

## Adversarial mutation campaign

The mutation suite changes one theorem-critical object at a time in a temporary copy and requires rejection. The seven cases are:

| Mutation                         | Intended gate                                |
|----------------------------------|----------------------------------------------|
| Lower-row endpoint               | exact interval and propagation semantics     |
| Final-fan radial numerator       | fan geometry and margin semantics            |
| Inherited S-procedure multiplier | exact multiplier consistency                 |
| Section-hull vertex              | byte-identical hull reconstruction           |
| Cap-allocation bracket           | certificate reconstruction and cap semantics |
| Disjointness threshold           | exact cross-axis separation gate             |
| Theorem-coefficient digit        | theorem-certificate byte identity            |

The archived transcript is `transcripts/adversarial_mutations.txt` and ends with:

```
MINVOL ADVERSARIAL MUTATION SUITE: PASS
```

These tests show that the selected failure paths are live. They are not a complete fault-injection proof for every line of code.

## Construction-time export equivalence

Before the public-format certificate was separated from the private research repository, ten theorem-critical fields were compared semantically against the source package:

- theorem coefficient;
- split radius;
- lower-row list;
- bottleneck row;
- final fan minimum;
- forced-core lower bound;
- cap total;
- near-Jung lower bound;
- handoff margin; and
- relevant fan census/fingerprints.

The result is archived in `transcripts/export_equivalence.txt`. The public package does not require the private repository to replay.

## Paper and documentation audits

`paper/reconcile_certificate.py` verifies that the manuscript uses generated certificate constants and that the lower-branch table is byte current. The Markdown/PDF pair checker requires exactly one committed PDF for every public Markdown source. The Poppler preflight checks readability, positive page count, font embedding, and extractable text.

Page-by-page visual inspection remains a human step. Automated PDF validity is not a mathematical proof check.

## Independence limitations

The Python and R audits are independent implementations, but they are not fully independent experiments:

- they consume records derived from the same certificate objects;
- they implement the same mathematical formulas;
- they were prepared within the same project;

- they have not been reviewed by an external subject-matter expert; and
- their agreement cannot rule out a common mathematical misconception.

Accordingly, the package should say **independent implementation audits**, not **independently verified theorem**.

## External audit still requested

The most valuable next audit would be an external reviewer who independently:

1. checks the geometric reduction in the manuscript;
2. reimplements at least one critical exact component;
3. reconstructs a selected lower row or cap pair directly from the stored arrays;
4. verifies the branch handoff and universal quantifiers; and
5. reports any discrepancy publicly with the exact release tag and commit.